

Task: 7

10 Marks

Lab Task 1

Design a Payroll class that has data members for an employee's hourly pay rate, number of hours worked, and total pay for the week. Write a program with an array of seven Payroll objects. The program should ask the user for the number of hours each employee has worked and will then display the amount of gross pay each has earned.

Input Validation: Do not accept values greater than 60 for the number of hours worked.

Lab Task 2

The objective of this exercise is to understand the declaration of a class with its data members and Member functions. Also, the concept of **passing objects** as function parameters.

Create a class **Travel** with two data members kilometer and hour, class member functions input, show and add, which accept an object of Travel as parameter. It adds the values of data members of the parameter object and the value of the calling object's data members and displays the result.

Declare two **objects t1** and **t2** of class Travel and input data in both objects. Call the add function with **t1 object** and pass **t2 object** as an argument.

Travel
km : int hr : int
Travel() ~Travel() input() : void show() : void Add(Travel p) : void

Lab Task 3

The objective of this exercise is to understand the concept of **const** object type arguments and arguments passing by reference.

Problem Statement

Write a class definition **ComplexNumber** with two data members' **realPart** and **imaginaryPart**. We wish to perform addition, subtraction and multiplication with objects of **ComplexNumber** class.

A complex number $c = a + bj$ consists of two parts, the real part **a** and the imaginary part **bj**. We can perform arithmetic operations (+, -, *, /, or addition, subtraction, multiplication) on complex numbers as follows:

$$\begin{aligned} +: (a1 + b1j) + (a2 + b2j) &= (a1 + a2) + (b1 + b2)j \\ -: (a1 + b1j) - (a2 + b2j) &= (a1 - a2) + (b1 - b2)j \\ *: (a1 + b1j) * (a2 + b2j) &= (a1a2 - b1b2) + (a1b2 + a2b1)j \end{aligned}$$

A UML diagram of this class is given as:

ComplexNumber
realPart : double imaginaryPart : double
Complex() ~Complex() Complex(double, double) add(const Complex&) : Complex sub(const Complex&) : Complex mul(const Complex&) : Complex print() : void const

Create objects of this class and test all the member functions.